Toward Efficient Eye Tracking in AR/VR Devices: A Near-Eye DVS-Based Processor for Real-Time Gaze Estimation

Shihang Tan, Jinqiao Yang, *Student Member, IEEE*, Jiayu Huang, Ziyi Yang, Qinyu Chen, *Member, IEEE*, Lirong Zheng, *Senior Member, IEEE*, and Zhuo Zou, *Senior Member, IEEE*

***Abstract*— This paper presents an efficient near-eye dynamic vision sensor (DVS)-based processor for real-time eye tracking in augmented reality/virtual reality (AR/VR) devices. The processor takes advantage of the sparse event data with fine time resolution from the DVS, addressing the need for high frame-rate, low-power, and accurate eye tracking on wearable devices with extended battery life. Exploiting the inherent sparsity of event data, we propose an event-density-based region of interest (ROI) determination method that operates directly on event stream, which requires 47× fewer operations than the traditional methods, effectively overcoming the latency problem caused by the heavy computational loads. To eliminate the issue of decreasing accuracy at the edges of the field of view (FoV), we customized and fine-tuned a neural network for gaze estimation, ensuring uniformly distributed sub-degree accuracy. An estimator with a streamlined output mapping strategy and an adaptive window-sliding convolution scheme is implemented for gaze estimation acceleration. The processor is designed and fabricated in UMC 40-nm LP technology with a core area of 2.52 mm^2 and performs end-to-end eye tracking exclusively with the raw event stream from DVS, achieving an average accuracy of 0.91° within a 96°×64° FoV. Operating at 200 MHz, it achieves a dynamic frame rate of up to 1.2 kHz and requires only 12.7 μJ of energy per gaze estimation. By integrating the DVS, the processor enables real-time, low-power, and accurate eye tracking, enhancing the immersive experience on AR/VR devices and offering intuitive and seamless interactions.**

***Index Terms*— Eye tracking, dynamic vision sensor (DVS), event-driven, neural network (NN), AR/VR device.**

I. Introduction

IN RECENT years, the field of Augmented Reality (AR) and Virtual Reality (VR) has witnessed remarkable advancements, revolutionizing the way we interact with our digital surroundings. At the heart of these immersive experiences lies a technology that has proven to be indispensable - eye tracking [1]. The basic illustration of an eye tracking system is shown in Fig. 1. It captures eye movement information and provides an estimated gaze point as output. Eye tracking in AR/VR environment can enable gaze-based interactions [2], allowing users to control and navigate virtual environments simply by looking at specific objects or areas of interest. It provides a natural and intuitive way of interaction, mimicking the way humans naturally explore their surroundings [3]. The importance of eye tracking in AR/VR devices has been recognized by researchers and developers, leading to ongoing efforts to implement this technology into wearable devices [4], [5], [6]. However, this endeavor presents significant challenges that must be overcome to ensure effective and reliable eye tracking performance [7].

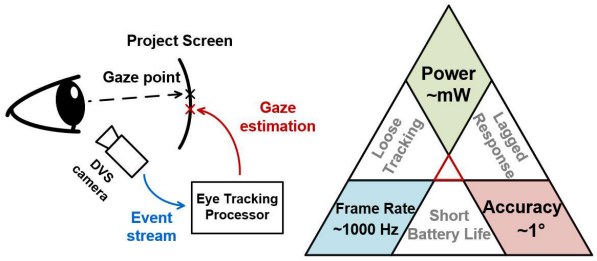


Fig. 1. An illustration of an eye tracking system in near-eye head mounted devices and its design concerns.

The primary design considerations for an eye-tracking system can be visualized as a trade-off triangle, as shown in Fig. 1. The three corners of the triangle represent the key objectives: (1) sub-degree accuracy, (2) kilo-hertz frame rate and (3) milli-watt level power consumption. Accurate gaze estimation is crucial for a seamless and natural user experience. To distinguish between closely spaced targets, such as adjacent icons or text, an eye tracking system must achieve high accuracy, ensuring the estimated gaze point is within a 1° radius of the actual focus for precise interactions. Sub-degree accuracy also enhances foveated rendering and varifocal displays [8], improving XR experiences and reducing computational load [9]. Moreover, the processing frame rate is also essential for real-time eye tracking. Rapid eye movements like saccades can reach speeds of up to 500°/second [10]. A frame rate of at least 1000 Hz can ensure smooth tracking and reduce motion sickness in virtual environment [11]. Additionally, low power consumption is critical for wearable devices. Eye-tracking systems must operate at low power

Received 18 September 2024; revised 2 March 2025; accepted 14 March 2025. Date of publication 1 April 2025; date of current version 1 October 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 92164301, in part by Shanghai Municipal Science and Technology Project under Grant 24TS1410500, and in part by Leiden University Global Fund (LUGF) Seed Fund 2025. This article was recommended by Associate Editor F. Z. Z. Rokhani. *(Corresponding author: Zhuo Zou.)*

Shihang Tan, Jinqiao Yang, Jiayu Huang, Ziyi Yang, Lirong Zheng, and Zhuo Zou are with the State Key Laboratory of Integrated Chips and Systems, School of Information Science and Technology, Fudan University, Shanghai 200433, China (e-mail: zhuo@fudan.edu.cn).

Qinyu Chen is with Leiden Institute of Advanced Computer Science, Leiden University, 2311 EZ Leiden, The Netherlands.

Digital Object Identifier 10.1109/TCSI.2025.3553497

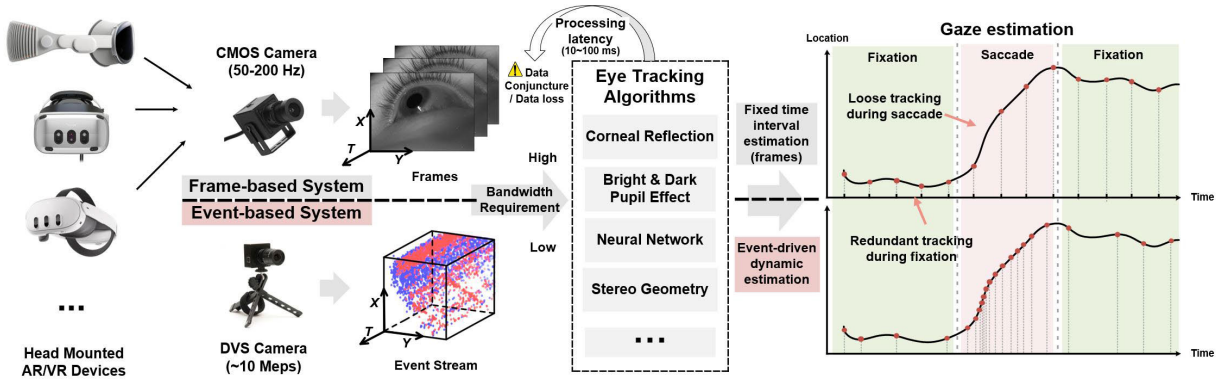


Fig. 2. Illustration of frame-based system and event-based system for eye tracking and their respective estimation patterns.

levels, typically in the milliwatt range, to extend battery life and enable prolonged usage. This requirement is especially challenging due to the computational intensity of continuous image processing.

Fig. 2 illustrates some of the most commonly used head-mounted devices (HMDs) and their corresponding eye tracking algorithms and systems, all of which are frame-based and utilize CMOS cameras as sensors. A recent study analyzed the latency of most available HMD eye trackers and reported a tracking latency ranging from 45 to 81 ms [12], which is far below the kilo-hertz frame rate requirement for accurately capturing eye movements and cause potential data congestion or data loss. Additionally, CMOS sensors that are capable of reaching 1000 Hz consume significant power, far exceeding the milli-watt level power budget, and the large amount of data generated also requires high bandwidth and substantial energy for transfer and processing, posing challenges for real-time applications on wearable devices. Besides, many frame-based systems claim to achieve sub-degree accuracy, but this accuracy is often limited to a narrow FoV [1]. Outside this area, the accuracy tends to drop noticeably.

Dynamic Vision Sensor (DVS) [13] offers a viable alternative to deal with these challenges. It's activated only by changes in the scene and generates sparse events with fine temporal resolution [14]. This feature has been applied in gesture recognition [15], autonomous driving [16], [17], and many other scenarios. The unique characteristics of DVS also make it highly suitable for high-speed and low-power eye tracking, as it generates less data and reduces processing needs during fixation while still capturing fast, subtle eye movements during saccades. The operating mechanism of the DVS also provides a high-contrast range, enhancing robustness under variable lighting conditions. This has been successfully demonstrated in other applications [18], and is expected to offer significant advantages for robust eye tracking in environments with substantial lighting variations. Early works using DVS for eye tracking have shown promising results [19], [20]. However, they did not fully exploit the intrinsic properties of event-based data, making it challenging to deliver the required high frame rate, low-power, and accurate eye tracking on mobile devices.

In this work, a DVS-based near-eye end-to-end processor aiming to fully exploit the unique characteristics of event data is proposed for efficient eye-tracking deployment on mobile AR/VR devices. It's exclusively utilizes event data from DVS and performs kilo-hertz gaze estimation with sub-degree accuracy and milli-watt level power consumption. The main contributions of this work are summarized as follows:

- An event-density-based region of interest (ROI) determination method for locating the eye position and Address-Event Representation (AER) data stream pre-processing is proposed and implemented. This method only requires 2.13 MOPs (47× less than [21]) and reduces the focused area to 1/3 of the original size.
- An estimator with the streamlined output mapping strategy and adaptive window-sliding convolution scheme is implemented for gaze estimation acceleration, improving the system's throughput by 2.7×. It's dynamically triggered by the pre-processing logic and can operate at 200 MHz, achieving a processing latency of less than 1 ms.
- The proposed eye tracking processor is fabricated with UMC 40-nm LP technology. It achieves a dynamic frame rate up to 1.2 kHz with an average angular error of 0.91° across a 96°×64° FoV and consumes 12.7 μJ per estimation.

The remainder of this paper is organized as follows: Section II introduces the related works on near-eye eye tracking system. Section III presents our DVS-based end-to-end system design. Section VI provides an overall architecture of the processor as well as detailed descriptions on the pre-processing unit and estimator implementation. Section V shows the processor validation and performance evaluation. Finally, Section VI draws a conclusion.

II. Related Works

A. Frame-Based Eye-Tracking Methods

Traditional frame-based eye tracking methods typically use CMOS cameras as image sensors, with two common approaches: model-based and appearance-based eye tracking. Model-based eye tracking uses computational models based on eye anatomy and geometry to estimate gaze direction [22],

[23]. These models assume certain aspects of eye structure, such as pupil position and corneal reflections [24], [25]. By analyzing geometric relationships, they achieve sub-degree accuracy [25] but often require manual calibration and struggle with variations in eye shape and lighting conditions.

Appearance-based eye tracking focuses on the visual appearance of the eye, using image processing and machine learning techniques [26]. This approach leverages features like pupil position [27] or raw eye images [28], [29], [30] to estimate gaze direction, often using large datasets to learn these relationships. While more flexible in handling variations, it requires substantial training data and the model can be computationally intensive and leads to large processing latency. Additionally, frame-based methods often require high-resolution cameras, which can be both expensive and cumbersome for mobile devices. Moreover, standard CMOS camera frame rates generally peak at 200 Hz. Cameras with higher frame rates consume significant power, often at watt-levels [31], which exceeds the milli-watt power budget for a mobile eye tracking system.

B. Event-Based Eye-Tracking Techniques

Event-based eye tracking utilizes Address-Event Representation (AER) data from DVS, achieving high frame rates with much less bandwidth compared to traditional frame-based solutions. Several works have combined frame and event data for eye tracking. The framework proposed in [19] is the first to use event data for eye tracking, combining frame and event data to achieve a 10 kHz tracking rate through geometric fitting techniques. [32] extends Angelopoulos's work by using a segmentation network and template-based tracking method to boost performance. Both of these methods highlight the benefits of combining event data with frame data to achieve a high frame rate and satisfactory accuracy. However, the challenges of power consumption and maintaining accuracy in non-central regions remain unaddressed.

On the other hand, some researchers have focused on using only event data for eye tracking [33]. Although many DVS can output both frame and event data simultaneously, the power consumption in event-only mode is much lower compared to outputting both events and frames.[1] Therefore, a purely event-based system can be significantly more energy-efficient than the hybrid systems or the frame-based system. In [36], a lightweight 2D-CNN model for gaze estimation using only event data was proposed, collected under similar settings as [19]. While this method avoids using hybrid event-frame data, it suffers from lower accuracy (9.974°) and a large time window, making it unsuitable for real-time scenarios. Chen's work in [37] introduces a Change-based Convolutional Long-Short-Term Memory (CB-ConvLSTM) model for event-based pupil tracking, reducing arithmetic operations by 4.7 times while maintaining accuracy. However, Chen's approach uses synthetic event data from RGB images and a fixed time window, which limits the advantages of event data and makes it difficult to achieve the kilo-hertz frame rate requirement.

These studies have provided valuable insights into the field of eye tracking technology, highlighting the advantages and limitations of various approaches. However, despite advancements in this area, few works have successfully addressed the challenges of developing an accurate, high-frame-rate, and energy-efficient eye tracking system. In this work, we present a fully event-based processor that incorporates an event-density-based ROI determination and a customized eye tracking network, along with event-driven hardware designs. This processor enables kilo-hertz gaze estimation with milli-watt level power consumption and sub-degree accuracy, making it highly suitable for eye tracking on mobile devices.

III. System Design

The proposed end-to-end DVS-based eye tracking system's working pipeline is shown in Fig. 3(a), which exclusively utilizes raw AER data to generate estimated gaze coordinates. It is worth noting that although the sensor's field of view may cover a larger area, the region that encompasses meaningful eye movement is often considerably smaller. So we established an event-density-based method for region of interest (ROI) determination to help the system focus on the specific eye region. For event-to-frame conversion, we use constant-event-count sample aggregation with an overlapping mechanism to transform the ROI-filtered events into feature maps. To achieve accurate eye tracking, we have developed a customized eye-tracking network, which has shown its capability to achieve sub-degree accuracy across the full FoV.

A. Event-Density-Based ROI Determination

To ensure that the sensor in a head-mounted device effectively tracks eye movements, it must adapt to various eye shapes and sizes. This often results in unused space within the sensor's field of view, meaning that significant eye activity is confined to a smaller area. To focus on this region of interest (ROI) and eliminate unnecessary data and noise, it is crucial to identify and concentrate on the active eye region. Traditionally, ROI determination has relied on segmentation networks [21], [38], which require converting event streams into frames before processing them through complex networks for eye area detection. This approach not only underutilizes the efficiency of neuromorphic event data but also leads to increased computational load and delays.

Instead of considering this problem from the perspective of frames, we propose an event-density-based ROI determination method that works directly on the event stream without needing an event-to-frame conversion process. In near-eye tracking setups, eye movements predominantly trigger events from DVS due to their asynchronous detection capabilities. Thus, the sensor naturally highlights active eye regions through high event density while other areas remain inactive. By focusing on regions with peak event density directly from the event stream, we simplify the identification of the ROI and avoid irrelevant data.

This method is detailed in Algorithm 1, where real-time event monitoring is maintained using continuously updated

[1]Sony IMX500 CMOS camera [34] consumes 278.8 mW at 30 FPS. CDAVIS [35] consumes 106-120 mW with both frame and event output enabled, and 15-32 mW when only the event output is enabled.

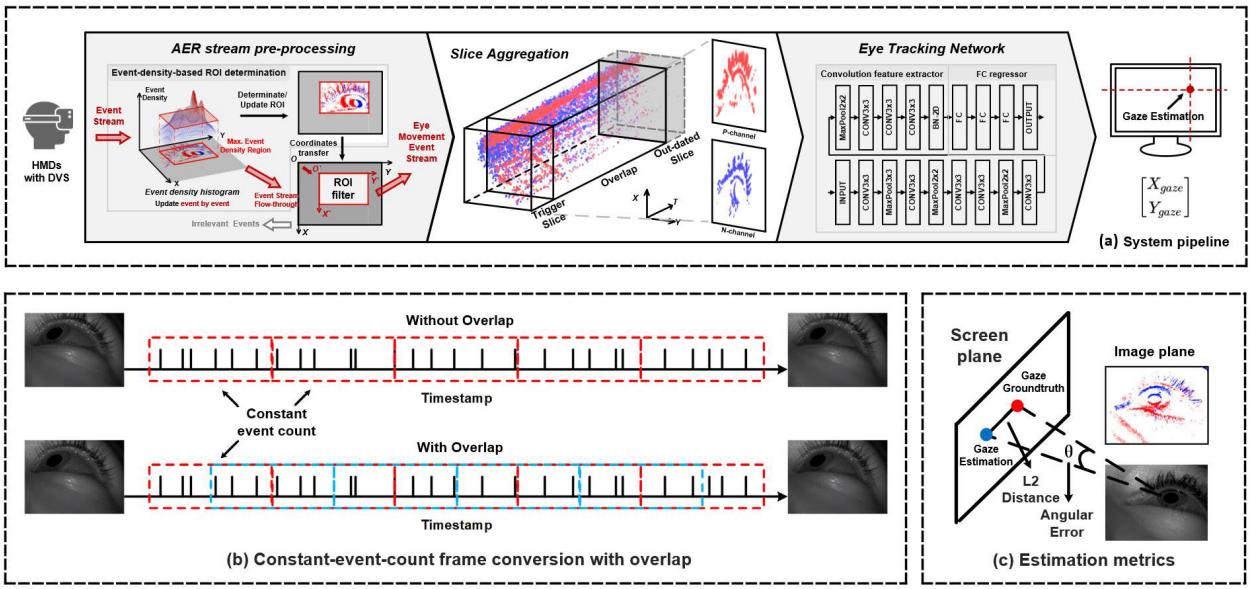


Fig. 3. (a): The overview of the proposed DVS-based end-to-end eye tracking system. (b): The constant-event-count event-to-frame conversion with overlap illustration. (c): The two estimation metrics: L_2 distance and angular error.

Algorithm 1 ROI Determination From AER Data Stream

Data: AER event stream, $\mathbf{h}$, $\mathbf{w}$ height and width of ROI, $\mathbf{H}$, $\mathbf{W}$ height and width of the frame

Result: $(\mathbf{x_b}, \mathbf{y_b})$ top left corner of the bounding box representing the ROI, $(\mathbf{x_{new}}, \mathbf{y_{new}})$ the event coordinates in the cropped coordinate system

Initialize `XIntervals` and `YIntervals` as arrays of zeros with lengths $\mathbf{H} - \mathbf{h} + \mathbf{1}$ and $\mathbf{W} - \mathbf{w} + \mathbf{1}$ respectively

for *each event* (x_i, y_i) *in the event stream* **do**
  for *each interval* $X_j = [j, j + h)$ *in* `XIntervals` **do**
    if $x_i \in X_j$ **then**
      increment `XIntervals[j]` by 1
    end
  end
  for *each interval* $Y_j = [j, j + w)$ *in* `YIntervals` **do**
    if $y_i \in Y_j$ **then**
      increment `YIntervals[j]` by 1
    end
  end
end
$x_b \leftarrow$ argmax(`XIntervals`)
$y_b \leftarrow$ argmax(`YIntervals`)
for *each event* (x_i, y_i) *in the following event stream where* $x_b \le x_i < x_b + h$ *and* $y_b \le y_i < y_b + w$ **do**
  $x_{new} \leftarrow x_i - x_b$
  $y_{new} \leftarrow y_i - y_b$
end

X/Y axis interval histograms. As each event occurs, candidate intervals are refined, and the ROI is determined or updated without accumulating intermediate frames. This approach aligns with the strengths of DVS by dynamically identifying active regions from the event stream, ensuring efficient processing. Being able to work directly on the event stream also means that subsequent processes in the system pipeline do not have to wait for the ROI results. In other words, the ROI process, which often tends to induce unaccountable latency in the system, now operates in a non-blocking fashion and induces minimal latency in the system pipeline. The DVS's intrinsic event-triggering mechanism also gives the algorithm great simplicity compared to segmentation networks. The combined advantages of being frame-free and low-complexity make this method responsive and less resource-intensive, yielding ROI determination and updates in real-time.

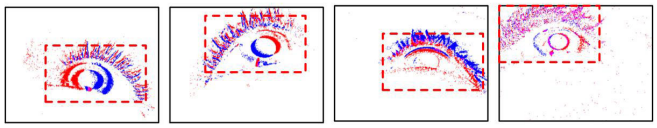


Fig. 4. The determined ROI with the proposed event-density-based method. In this case, we used 8000 events for each determination. The height and width of ROI are fixed for each subject.

Fig. 4 shows some of the determined ROI using the proposed event-density-based method. In this system, we selected the size of ROI as 120 × 220 because most subjects in the dataset exhibited eye movements within a bounding box of 120 × 220 pixels under the sensor's resolution (346 × 260). Additionally, considering the event rate of the DVS sensor, we determined that an event threshold of 8000 is optimal for robust ROI determination and updating, which balances the need for sufficient spatial density capture while minimizing latency. Once the ROI is established, the region can be utilized to filter out unwanted events occurring outside its boundaries, allowing the system to focus solely on the eye region. The filtered event stream will then be aggregated into slice frames in the next stage of the processing pipeline.

B. Event Slice Aggregation

To facilitate subsequent analysis and processing, the event stream passing through the ROI logic is converted into

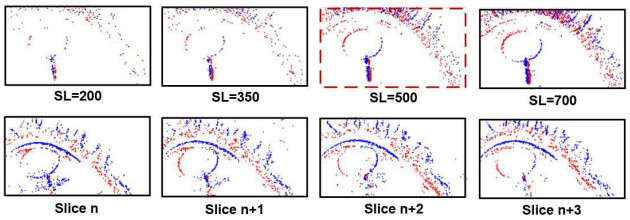


Fig. 5. The first row is event slices with different slice length (SL). The second row is a consecutive series of event slices with the length of 500 and 0.6 overlap.

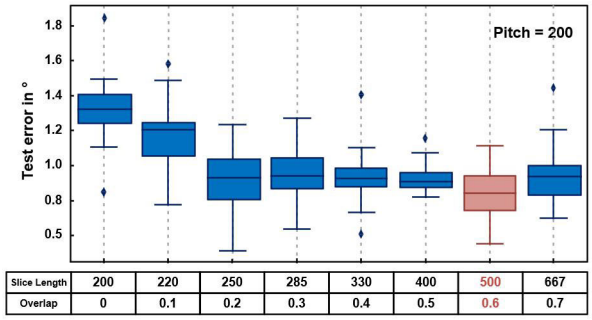


Fig. 6. The box plot of estimation angular error under different event slice length and overlap configuration. The pitch is set as 200 through the different settings.

frames, ensuring a consistent representation over time. Similar to the event-to-frame conversion method described in [39], we choose to use the number of events as the criterion rather than a fixed time window. Specifically, each sample is formed by aggregating a constant number of consecutive events. This approach is chosen because the number of events generated by the eyes during fixation and saccades can vary significantly within a given time window. If we were to accumulate all the events within a fixed time window to form a sample, it would lead to feature prominence inconsistencies across samples, making the subsequent learning process more challenging.

To enhance sample features and achieve higher frame rates, we used an event overlap mechanism to connect adjacent samples, which is illustrated in Fig. 3(b). This mechanism effectively increases the number of events in a sample, making the features more prominent. It also reduces the events needed for new inferences, improving the theoretical frame rate. The event overlap mechanism captures dynamic features by smoothing information flow and reducing discontinuity between samples, enhancing accurate feature extraction and analysis of eye-tracking data.

Fig. 5 shows the frames with different event slice length and overlap, illustrating the importance of choosing the optimal event slice length (SL) and overlap (OV). The number of events required to trigger a new estimation is determined by the equation:

$$pitch = SL \times (1 - OV) \tag{1}$$

If the pitch value is too high, the system would require more events to generate a new estimation, resulting in a lower practical frame rate. Conversely, if the value is too low, it either means a smaller event slice length or a larger overlap. The former can lead to insignificant features, making the estimation more difficult, while the latter could make adjacent samples too similar, leading to overfitting and potential biased learning.

To determine the optimal combination of event slice length and overlap, we conducted a series of experiments. The event rate of the human eye in a near-eye setting is generally around 200 per millisecond during saccades [19]. To achieve a kilohertz-level frame rate and capture saccadic movements in real-time, the system should be able to trigger a new estimation every 200 events. With this value determined, we chose different sets of parameters for our experiments. In these settings, the pitch value is fixed at 200. Fig. 6 shows the different configurations and the corresponding performance in accuracy. The samples with a 500 slice length and an overlap of 0.6 outperform other configurations. There is a correlation between increasing the overlap and the slice length with the performance. However, the overlap larger than 0.6 generates overly similar samples and causes the aforementioned overfitting problems on test set. The second row in Fig. 5 displays a sequence of consecutive frames with a slice length of 500 and an overlap of 0.6. These frames contain sufficient features, while the differences between adjacent frames are distinct, effectively illustrating the movement trend.

C. Eye Tracking Network

To overcome the fragility of geometric models in handling diverse eye shapes and environmental conditions, we have opted for a neural network model for gaze estimation, aiming to achieve uniformly distributed accuracy across the full FoV. Furthermore, to address latency and energy consumption issues associated with the high computational loads of standard models [21], [29], we have developed a customized lightweight estimation network and manually optimized its structure to adapt for our system design and enhance efficiency.

1) Network Structure: The customized eye-tracking network takes a $2 \times 120 \times 220$ pre-processed feature map as input and outputs gaze point coordinates. It consists of 8 convolution layers, 4 max pooling layers, a 2D Batch Normalization layer, and 3 fully connected (FC) layers. Each convolution layer uses a 3×3 kernel with a stride of 1 and Rectified Linear Unit (ReLU) activation. Max pooling layers follow the first, second, fourth, and fifth convolution layers, with varying kernel sizes and strides. After the last convolution layer, 2D batch normalization stabilizes the FC computation. The activations are flattened and passed through the FC layers, with the final layer outputting the gaze coordinates. The eye tracking network is implemented with Pytorch [40] and comprises a total of 91k parameters. During inference, the network requires only 11.5M MAC operations to complete the forward pass and generate the gaze point prediction. Table I presents a comparison between our eye tracking network and related algorithms, highlighting significantly lower Floating Point Operations (FLOPs) and parameter numbers.

2) Network Training: The training and testing sets are created without distinguishing between the left and right eye. However, since the collected data includes information from both eyes, we apply a horizontal flip to the event coordinates of the right eye data. This effectively doubles the training samples

TABLE I
FLOPS COMPARISON WITH THE BEST METRICS IN **BOLD**

Method	Task	#FLOPs (M)	#Params (M)
2023 CB-ConvLSTM [37]	Pupil tracking	214	0.418
2024 TennSt [42]	Pupil tracking	5940	0.81
2023 FBNet-C100 [21]	Gaze estimation	120	3.59
Ours	Gaze estimation	**23**	**0.091**

and reduces hardware costs compared to a binocular system, as it requires only one sensor. The training was conducted on an NVIDIA GeForce 3080 for 200 epochs with a batch size of 64. We employed the ADAM [41] optimizer with a ReduceLROnPlateau scheduler. The initial learning rate was set to 0.01, and it was reduced by multiplying a scale factor of 0.3 when the loss on testing set did not decrease for 5 consecutive epochs.

3) Quantization: Although quantization plays a crucial role in practical deployment on hardware, it is essential to take into account that the ground truth values cover a 1080p 40-inch screen. This requirement necessitates activations in the network to have an adequate bit width to accurately represent these coordinates. To effectively reduce the memory footprint and computational demands of the proposed eye tracking network without compromising its performance, we implemented post training quantization (PTQ) with a customized quantization function which converts the floating-point numbers into fixed-point numbers. Specifically, we cast the floating-point weights into 8-bit fixed-point numbers and the intermediate activations into 20-bit numbers. This approach allows us to achieve the desired optimizations for hardware deployment while maintaining the network's performance.

4) Metrics and Evaluation: There are two commonly used metrics to evaluate the accuracy of eye tracking, which are shown in Fig. 3(c). One of these metrics is the L_2 distance, which serves as a primary metric for evaluating the accuracy of 2D gaze estimation. The L_2 distance is defined as the mean Euclidean distance between the gaze predictions and their respective ground truth gaze annotations. It can be obtained from Eq. 2, where (x_{gt}, y_{gt}) refers to the ground truth coordinates of the gaze point, and (x_{pred}, y_{pred}) refers to the prediction of the gaze point.

$$L_2 = \sqrt{(x_{gt} - x_{pred})^2 + (y_{gt} - y_{pred})^2} \tag{2}$$

Angular error, defined as the angular difference between the predicted gaze point and the ground truth, is another key metric for assessing gaze estimation accuracy. It accounts for the subject's distance from the screen, offering a more meaningful evaluation across different viewing conditions. The calculation is represented by the equation:

$$E_{ang} = \arccos\left(\frac{\boldsymbol{G_{gt}} \cdot \boldsymbol{G_{pred}}}{|\boldsymbol{G_{gt}}|\,|\boldsymbol{G_{pred}}|}\right) \tag{3}$$

Here, $\boldsymbol{G_{gt}} = (x_{gt}, y_{gt}, L_0)$ and $\boldsymbol{G_{pred}} = (x_{pred}, y_{pred}, L_0)$, where L_0 represents the distance between the subject and the screen onto which the gaze point is projected.

As the samples are aggregated with overlap, we carefully divide the AER stream into training and testing sequences to prevent data leakage. These sequences are then further processed into training and testing samples. Since we constructed our training and testing process to precisely mimic the AER data stream into the event-density-based ROI logic and the network, the trained network can adapt to the pre-processed features that have gone through the ROI determination. This joint optimization of the event-density-based ROI method and customized neural network achieves the same best-case accuracy and even slightly better average accuracy across subjects while providing greater flexibility for eye autonomy compared to previous work [43]. On average, the full-precision eye tracking network achieved an estimation error of 0.92° (14.3 pixels), with the best-case accuracy being 0.81° (12.5 pixels). In comparison, the quantized network exhibited an average estimation error of 1.15° (18.3 pixels), with the best-case accuracy being 0.91° (14.2 pixels). In addition, we conducted experiments on different FoV settings to assess the robustness of the model's performance. As illustrated in Fig. 7, while the model in EVBEYE [19] exhibits an increasing error trend as the FoV expands, our model shows a consistent accuracy distribution across the entire FoV with lower estimation error.

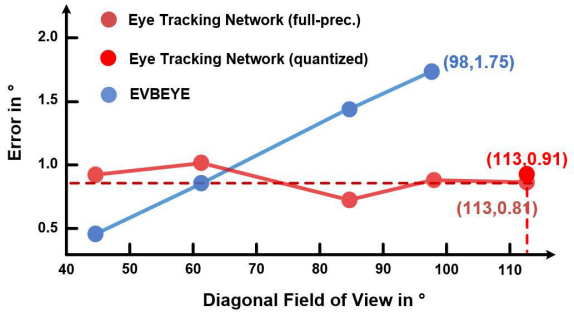


Fig. 7. The accuracy comparison of [19] in different FoV with the eye tracking network. The evaluations of both algorithms were conducted on the best-performing subject.

IV. PROCESSOR ARCHITECTURE

To meet the real-time, low-power, and high frame rate requirements for eye tracking on wearable devices, we designed a dedicated processor specifically tailored to support the methods proposed in Section III. The hardware architecture of the processor, as shown in Fig. 8, consists of an event-density-based ROI determination unit with an AER stream interface, an NN-based estimator, SRAM for feature map and weight storage, a power management unit, and a top controller. The input AER data are fed to the pre-processing unit, where the ROI is determined using the event-density-based method and then used to filter out irrelevant events. The events that fall into the ROI are aggregated to form a feature map. Once the number of newly input events reaches a threshold, the controller triggers the NN-based estimator to perform an inference. Within the estimator, the activation and weight data are re-organized and distributed to the PE array for computation through a data dispatcher, resulting in estimated gaze point coordinates. Clock gating is widely employed throughout the processor to minimize power consumption. Thanks to the sparsity-aware convolution

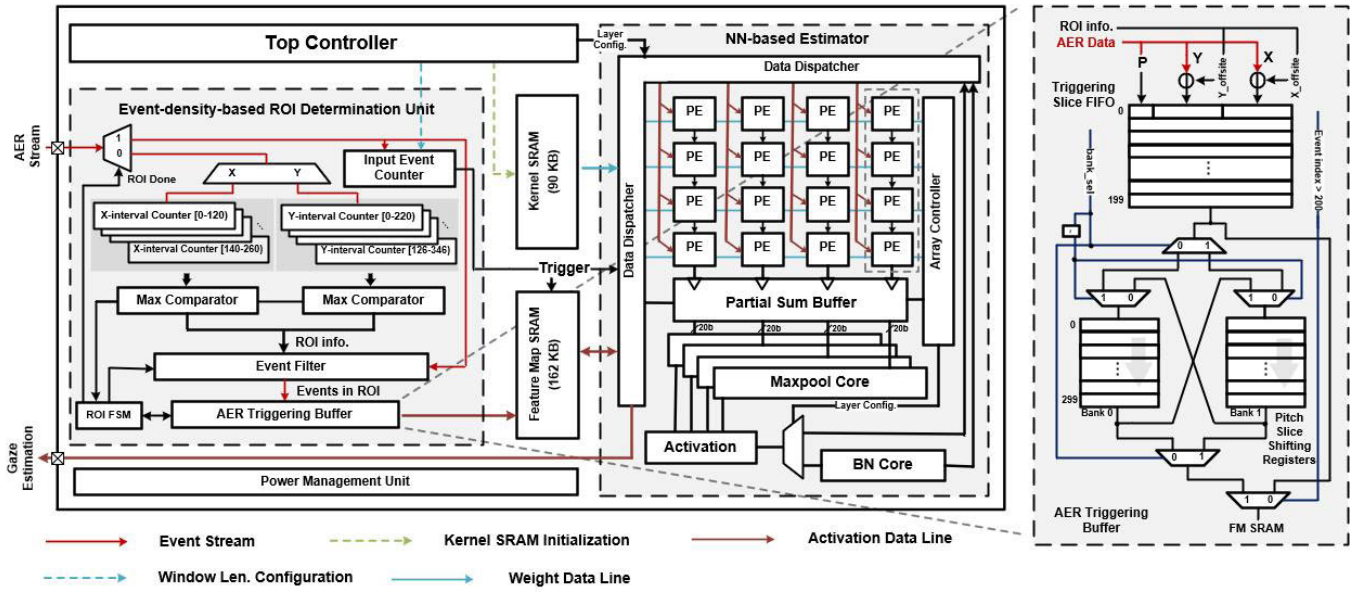


Fig. 8. The hardware architecture of the proposed DVS-based end-to-end eye tracking processor.

units, adaptive quantization circuits, and streamlined output mapping (SOM), the processor achieves significant data reuse and reduction in memory bandwidth, addressing the imbalance between computational parallelism and memory access. Additionally, all weights and activations required for inference can be stored on-chip due to quantization, allowing the processor to perform gaze estimation without off-chip memory access, further enhancing energy efficiency.

A. ROI Determination and Pre-Processing

Fig. 8 illustrates the hardware block as well as the data and control signal paths of the ROI determination and pre-processing logic. During pre-processing, the AER data is streamed through the pre-processing unit to identify the ROI. After determining the ROI, the AER data is transferred to the cropped coordinate plane and stored in the event accumulation buffer, where each event's coordinates map to a specific memory address, forming a 2-channel feature map. When enough new events are generated, the controller triggers the estimator to perform inference.

1) Hardware-Optimized Approach for ROI Determination: To implement the event-density-based ROI determination method described in section III-A, we use counters to represent the event intensity of intervals on each axis and comparators to identify the index of the counter with the largest value. Taking the X dimension as an example, the implementation employs a set of counters, denoted as X-interval Counter $[0, H_{ROI})$, Counter $[1, H_{ROI}+1)$, and so on, up to the last one $[260-H_{ROI}, 260)$. Here, H_{ROI} represents the length of the interval, which corresponds to the height of the ROI. These counters store the cumulative sums of events within specific ranges. By dividing the input data into these pixel ranges, we can accurately track the distribution of events along the X dimension without storing the entire feature map, thereby saving significant memory and computational resources.

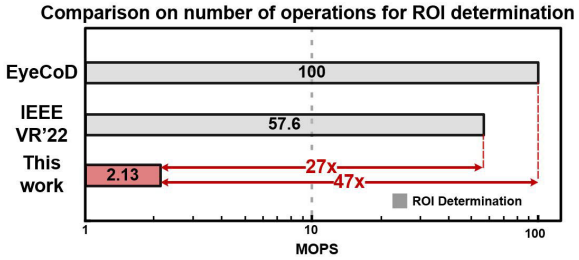


Fig. 9. The comparison between our event-density-based ROI determination method with related works [21], [44], focusing on the number of operations required.

In our specific implementation, we utilize 8000 events for ROI determination. The bounding box dimensions are set to a width of 220 and a height of 120. Given the original resolution of the DVS sensor is 346 by 260, the number of intervals on each axis would be 126 for the width and 140 for the height. Even in the worst-case scenario, where every event triggers all counters to accumulate, which is highly improbable, the number of addition operations required for the ROI process would be $8000 \times (126 + 140)$, resulting in a total of 2.13 million operations (MOPs). Fig. 9 compares our event-density-based ROI determination method with the common segmentation network solution in terms of the number of operations. By exploiting the unique characteristics of DVS and event data, our solution requires significantly fewer operations. Additionally, since it operates directly on the event stream, it introduces minimal latency into the processing pipeline. After the initial determination, the ROI can be updated without any delay as the events continuously stream through the determination logic, unlike the frame-based systems with segmentation network, which can significantly increase processing time and affect the overall end-to-end delay [21]. Compared to [43], while both approaches are event-driven, this method completely eliminates intermediate spatial grid buffering. The pre-processing unit continuously

refines ROI boundaries without pausing to accumulate or store intermediate representations. This not only reduces hardware resource requirements but also achieves even lower latency through a non-blocking processing approach. In this approach, the estimation process does not have to wait for ROI updates, thereby enhancing overall system efficiency.

Additionally, the hardware design offers configurability, allowing the size of the bounding box and the number of events used for the ROI process to be adjusted according to different situations. These flexible system parameters emphasize user- and scenario-specific configurability to address the diverse needs of AR/VR applications, and allow for dynamic parameter tuning for individual differences. For example, the ROI size can be adjusted to accommodate variations in inter-pupillary distance and eyelid geometry, while the event threshold is configurable to adapt to different blink rates or saccadic dynamics. In high-noise environments, thresholds can be increased to filter out spurious events, optimizing for both noise reduction and true eye movement detection. This flexibility expands the support for various specific application demands, such as prioritizing energy efficiency by reducing ROI size or enhancing precision by expanding it, ensuring tailored performance for varying user anatomical differences and application requirements.

2) AER Pre-Processing and Aggregation: Once the eye region is determined, the system needs to process subsequent input events. For each incoming event, it is first checked to see if it falls within the boundaries of the eye region. This boundary check ensures that only relevant events are further processed. If an event is found to be within the boundaries of the eye region, its coordinates will be transformed into the new plane of the ROI by subtracting the coordinates of the top-left corner of the bounding box, as shown in the first stage illustration in Fig. 3(a).

The transformed events will then be sent to the AER buffer, which triggers inference when the number of newly generated events reaches the pitch threshold. The structure of this buffer is illustrated in the right part of Fig. 8. It primarily comprises a triggering slice FIFO and two banks of shifting registers serving as a sequential buffer. This design can ensure dynamic triggering of subsequent computation modules and create the overlapped samples with no extra time lag.

To balance memory bandwidth and area cost, the 2-channel feature map is divided to store into four banks of feature map SRAM, allowing for parallel fetch and computation. Overlapping the halves during fetching slightly adds complexity to memory mapping but ensures computation efficiency. So the line with x coordinates equal to 60 is stored twice in both positive and negative maps for this purpose and zeros are padded at the beginning and end of each line.

B. NN-Based Estimator Structure

The NN-based estimator is responsible for the main computational load of the processor. Its hardware structure, shown in Fig. 8, includes a data dispatcher for feature map retrieval and reorganization, a reconfigurable PE array that handles computations for both convolutional and FC layers, a partial sum buffer, 4 max-pool cores for each PE line, and a BN core.

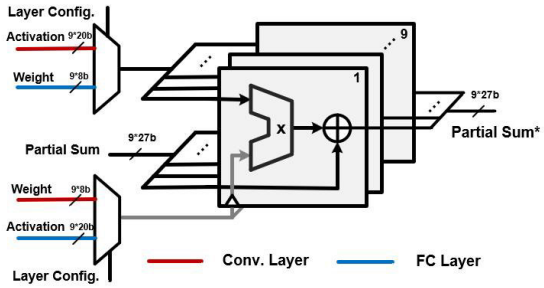


Fig. 10. Structure of a PE.

1) Weight Stationary Flow With Activation Reuse: After the frame aggregation is completed, the resulting $2 \times 120 \times 220$ feature map is re-arranged and sent to the PE array by the data dispatcher in the NN-based estimator. The structure of the PE array is shown in Fig. 8, which consists of 4 configurable PE lines. Within each PE line, there are 4 PEs, and every PE contains 9 MAC units, which is shown in Fig. 10. To optimize memory access and reduce the memory cost associated with storing partial sums, the system utilizes a weight stationary data flow approach [45], combined with an activation reuse strategy and a partial sum buffer made of register arrays.

The computation of each PE line begins by fetching 6 activation elements from SRAM, with three zeroes padded by the data dispatcher. In the subsequent cycles, as the window slides, the overlapped activation elements of two adjacent windows will be reused. Only the three new activations will be fetched from SRAM. In the case of the other 3 PE lines, they are loaded with weights for different output channels. The input feature map is broadcasted to each column, enabling the reuse of activations across different PE lines. In a stable state, the PE array can utilizes all 144 MAC units with a memory bandwidth of 12 elements. The combination of these reuse strategies proves effective in minimizing memory access and the associated costs.

2) Streamlined Output Mapping Strategy: As shown in Fig. 8, each PE line is composed of 4 PEs. The decision to cascade 4 PEs in one PE line is primarily based on the limitations of the feature map SRAM bandwidth. So, one batch of data could pass through four cascaded PEs and still be a partial sum. Storing these partial sums back to the feature map SRAM will incur additional memory footprint and accessing energy consumption. However, utilizing a longer PE line capable of completing the entire accumulation of one output element would not be fully optimized due to the restricted memory bandwidth, resulting in under-utilization of the computing hardware.

To balance the trade-off, we implemented a streamlined output mapping (SOM) strategy, optimizing hardware utilization while minimizing memory footprint and energy consumption. This strategy organizes the input feature map via the data dispatcher to align with output computations, prioritizing a single output result to reduce partial sums. For deployment, we designed configurable working patterns for each PE line and a partial sum buffer, as shown in Fig. 11. For example, when the input feature map exceeds four channels, it is divided into sets of four channels per PE line, each corresponding to

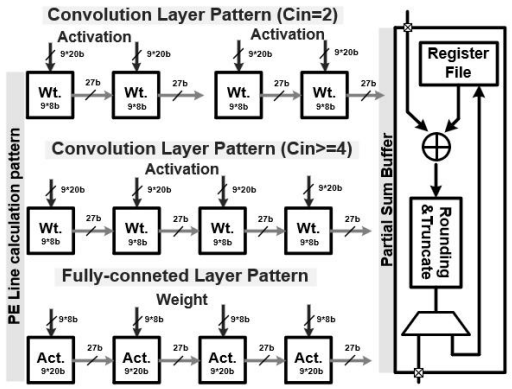


Fig. 11. Different computation patterns of one PE line, and the structure of partial sum buffer.

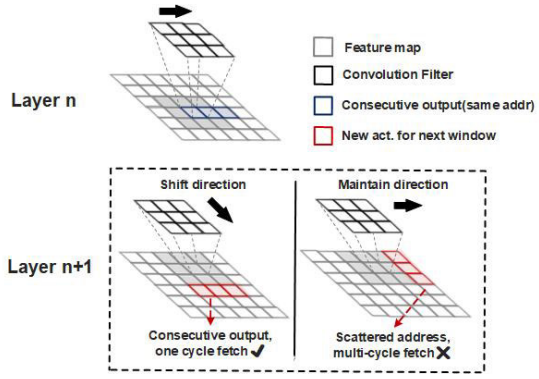


Fig. 12. The adaptive window-sliding convolution (AWC) scheme is illustrated to demonstrate how dynamically shifting the direction of window sliding can facilitate memory access.

the loaded weight. Input data forms a one-by-one convolution window, and intermediate results pass through four PEs. After processing a full row of four channels, weights are refreshed, and the next four channels are inputted. With this strategy, the generated partial sums are correlated and their number is reduced, allowing them to be stored in a buffer made of a register array and can be added in place in a single cycle between each weight refresh. During FC layer computation, PE lines are reused in different patterns as shown in Fig. 11. Activation values are loaded into each PE instead of weights, which are inputted every cycle between activation refreshes.

3) Adaptive Window-Sliding Convolution: The SRAM bank for each of the four feature maps has a width of 60 bits, allowing for a maximum of 12 activations to be written or read in a single cycle. However, a challenge arises when dealing with the output of the PE array, as it is arranged consecutively either by row or column, depending on the input order. Storing the data according to the PE array's output order can lead to a problem during the computation of the next layer. In this case, the required data will be scattered, requiring additional cycles to fetch them and significantly reducing the efficiency. To optimize this issue, we adaptively change the direction of the window rolling during convolution computation. The illustration of this scheme is shown in Fig. 12. For example, if the window sliding direction of the current layer is horizontal, it results in the output order of the feature map being row by row. Consequently, the results are stored back into SRAM with horizontally adjacent elements. Then, during the computation of the next layer, the window sliding direction is changed to vertical. Considering the reuse of the activations that overlap during the adjacent convolution window, the fresh activations that are inputted each cycle are horizontally adjacent, which matches their storage layout pattern. As a result, they can be fetched in a single cycle. The scheme of adaptively shifting window-sliding direction during convolution effectively reduces the complexity of the memory access pattern and overall computational delay while inducing a minimum amount of additional hardware resources.

TABLE II
ABLATION STUDY TO ASSESS BENEFIT OF SCHEMES FOR THE ESTIMATOR

System†	Throughput (FPS)	Energy/esti. (μJ)	Normalized Efficiency
Estimator	186.5	26.71	6.98 (1×)
Estimator with Act. reuse	442.1	20.82	21.23 (3.04×)
Estimator with Act. reuse and SOM*	751.6	17.24	43.60 (6.25×)
Estimator with Act. reuse and SOM and AWC**	1252.6	16.10	77.80 (11.15×)

† Weight stationary flow is used in all the settings.
* SOM: Streamlined Output Mapping strategy.
** AWC: Adaptive Window-sliding Convolution scheme.

To evaluate the effectiveness of the proposed schemes, we conducted an ablation study where we isolated them from the estimator. We performed a schematic-level analysis using inference test case waveforms to obtain the cycle count necessary for calculating throughput and generating Switch Activity Interchange Format (SAIF) files. These SAIF files contain the toggle rates of signals in the system during inferences. The SAIF files were then input into PrimeTime PX (PTPX) to measure power consumption in each scenario, which was further converted into energy per estimation. The simulations utilized UMC 40 LP-CMOS technology with operational conditions set at a voltage of 0.99 V, frequency of 200 MHz, under a worst-case corner scenario of 125 degrees to prove the system robustness under critical conditions and mitigate overoptimism. Normalized efficiency was calculated by considering both throughput and energy consumption as follows:

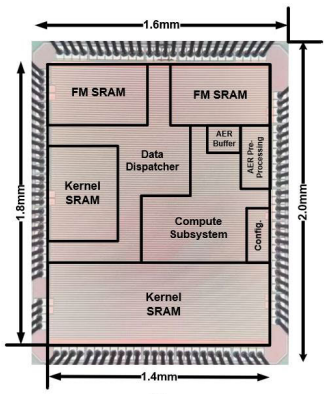
$$\text{Normalized Efficiency} = \frac{\text{Throughput (FPS)}}{\text{Energy per Esti. } (\mu\text{J})} \quad (4)$$

As shown in Table II, the combination of activation reuse, SOM and AWC scheme effectively improves the overall efficiency of the estimator by over 11 times. Additionally, it also reduces the memory footprint for storing partial sums and addresses the imbalance between the computation array and bandwidth of the memory, jointly resulting in inference latency of less than 1 ms.

V. EXPERIMENTAL RESULTS

A. Chip Implementation

Fig. 13(a) depicts the micrograph of the proposed chip, which is fabricated in a UMC 40-nm CMOS technology,



(a)

Technology		UMC 40-nm LP CMOS
Area	Core	1.8mm × 1.4mm
	Chip	2.0mm × 1.6mm
Gate Count (w/. | w/o. SRAM)		2.25M | 0.64M
Number of MACs		144
On-chip SRAM		252.19 KB
Core Voltage		0.7 - 1.0V
Clock Frequency		10 - 200 MHz
Power		15.2mW @ (1.0 V 200 MHz) 1.2mW @ (0.7 V 10 MHz)
Peak Power Efficiency		2.56 TOPS/W @ (0.8V 100 MHz)
Frame rate		1252 FPS @ (1.0 V 200 MHz)
Energy per estimation		9.0 μJ / infer. @ 100 MHz 12.7 μJ / infer. @ 200 MHz

(b)

Fig. 13. (a) Micrograph of fabricated chip. (b) Chip summary.

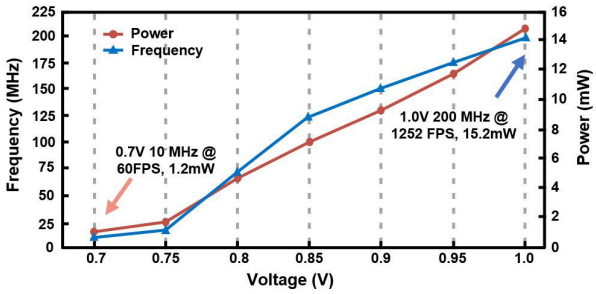


Fig. 14. Measured voltage frequency scaling of the chip.

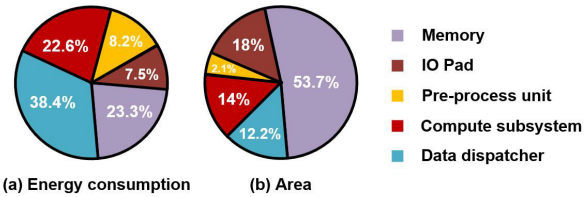


Fig. 15. Energy consumption and area breakdown of the processor. The compute subsystem is the estimator without data dispatcher.

and Fig. 13(b) provides a summary of the chip. The chip integrates 252.19 KB of on-chip SRAM within a core area of 2.52 mm^2 and a chip area of 3.2 mm^2. Fig. 14 shows the voltage frequency scaling of the chip. For the EVBEYE dataset [19], this chip can operate at 200 MHz frequency and triggers estimation in an event-driven manner, delivering a dynamic frame rate up to 1.2 kHz. The energy consumption for each estimation is 12.7 μJ. The chip can also be scaled down to 0.7 V and operate at 10 MHz, resulting in a power dissipation of only 1.2 mW. The area and power breakdown of the chip are shown in Fig. 15.

B. Testing Platform Setup

For valid and realistic benchmarking, we selected the dataset from [19], which is the first event-based dataset for eye tracking. During data acquisition, the user's head was secured on an ophthalmic headrest to minimize movement, while two DAVIS346 event cameras were positioned near each eye to accurately capture their movements. Stimuli were displayed on a 40-inch, 1920×1080 pixel monitor at a standard reading distance of 40 cm, ensuring that the testing closely reflects real-world scenarios. Additionally, the dataset employed

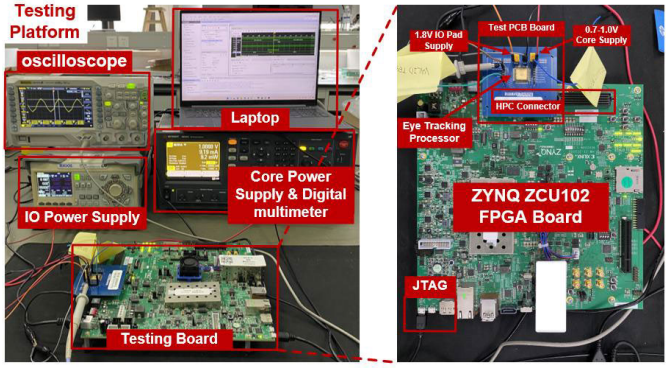


Fig. 16. Testing platform of the fabricated chip.

Near-Infrared (NIR) illuminators, aligning with key references in the field such as [28] and [46]. NIR illumination is imperceptible to the human eye, which minimizes distractions and supports natural behavior during testing. This type of illumination enhances the contrast of eye features, which is crucial for accurate tracking. Additionally, it helps mitigate the effects of ambient light variations, ensuring more reliable performance under diverse lighting conditions.

Figure 16 illustrates the testing setup for the fabricated chip, which includes a Xilinx ZYNQ ZCU102 FPGA Board, a test Printed Circuit Board (PCB) with the chip, a DC power supply, a digital multimeter, an oscilloscope, and a laptop. The FPGA manages the test event streams and the pre-trained model for chip configuration. Communication between the chip and FPGA occurs via the FPGA mezzanine connector (FMC), facilitating the transfer of data and control signals. The chip's estimation output is sent to the FPGA, which relays it to the laptop for further analysis. The DC power supply provides a core voltage of 0.7 V to 1.0 V and an I/O pad voltage of 1.8 V to 3.3 V, ensuring stable and reliable testing conditions.

C. Eye Tracking Performance

The system's triggering rate, which considers the actual event rate of eye data and balances it with the hardware processing delay, ensures no data loss. The DVS eye tracking processor triggers an inference every 200 events, approximating the event rate per millisecond during saccadic motion of human eye. With a processing latency of under 1 ms, it achieves a dynamic eye tracking frame rate of over 1.2 kHz. Extracting testing event slices from the raw AER data, we conducted verification for each subject, as shown in Fig. 17. On average, the processor achieves an estimation error of 1.15° (18.3 pixels) across all subjects, with the best case being 0.91° (14.2 pixels).

D. Comparison With the State-of-the-Art

Table. III compares our design with the state-of-the-art near-eye tracking systems. Compared to [43], this work achieves end-to-end (ROI and inference) latency equivalent to the former work's inference-only latency, while consuming 64% lower energy per estimation and maintaining the same sub-degree accuracy across the full 96°×64° FoV. While [19] and [32] achieve a higher theoretical frame rate, they did not report specific hardware platform details and have an

TABLE III
COMPARISON WITH STATE-OF-THE-ART EYE TRACKING SYSTEMS

	Input	Prediction	FoV	Method	Hardware Platform	Accuracy	FPS (Hz)	Energy/Esti.
This work	**Event**	**Gaze**	**96° × 64°**	**Customized NN**	**ASIC @ 200 MHz**	**0.91°**	**1252**	**12.7 μJ**
[43]	Event	Gaze	96° × 64°	Customized NN	ASIC @ 200 MHz	0.91°	1250 †	20.82 μJ
[19]	Frame/Event	Gaze	96° × 64°	Model-based	-	3.9°	10000 **	-
[20]	Event	Gaze	48 × 85°	CV+RNN	RTX3070 @1.5 GHz	0.46°	950	0.23 J *
[32]	Frame/Event	Gaze	96° × 64°	Template Match	-	4.7°	38.4k **	-
[47]	Event	Pupil	-	SNN	Speck SoC	3.24 px	<180	>16.06 μJ
[48]	Event	Corneal glint	-	CDL	-	<0.5 px	1000	>100 μJ
[49]	Frame	Gaze	40° × 40°	MobileNetV2	ASIC @ 115 MHz	3.16°	253	91.49 μJ
[44]	Frame	Gaze	40° × 40°	DNN	Jetson Xavier @1.37 GHz	0.5°	30	0.67-1 J *
[50]	Frame	Gaze	40° × 40°	GRU	Snapdragon 845 @2.8 GHz	3.65°	48	61.7 mJ *

* Calculation based on the datasheet of the hardware platform. ** Pupil fit updates only.
† Inference only, without the latency of ROI determination and pre-processing.

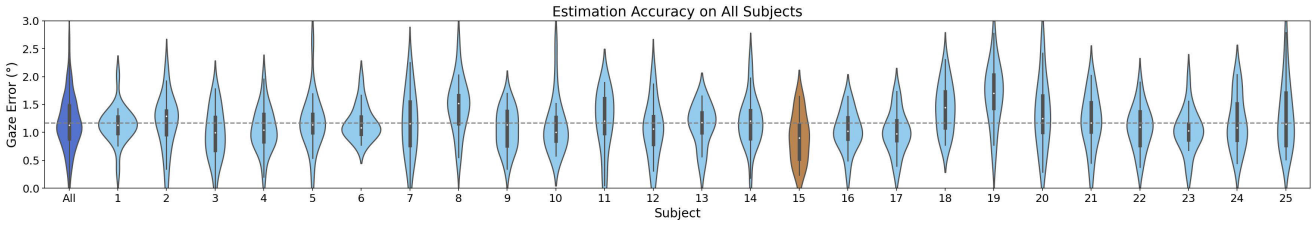


Fig. 17. The violin plot displays the processor's estimation error across different subjects, with the dotted horizontal line indicating the average accuracy.

average accuracy on the whole FoV that is several times lower. The reference [20] employs a geometrical method to extract ROI-related information and feeds it into a Recurrent Neural Network (RNN). This approach achieves a best-case accuracy of 0.46° after calibration. However, their system requires powerful GPUs to match the latency performance of our solution. Additionally, deploying their method on mobile devices is challenging due to hardware resource constraints and power consumption issues. Meanwhile, [44] also have accuracy below 1°, but their systems either operate within a smaller FoV and require significantly higher energy consumption per gaze estimation due to the lack of dedicated hardware design. Reference [49] utilizes FlatCam with an eye detection and gaze estimation model for gaze tracking. They co-designed a dedicated processor, achieving a frame rate of 253 Hz with the ASIC operating at 115 MHz. However, their algorithm is computationally heavy, with accuracy exceeding 3 degrees and energy consumption per estimation several times higher than ours. Reference [47] utilizes a Spiking Neural Network (SNN) for pupil tracking and deploys it on the neuromorphic System on Chip (SoC) Speck, achieving an accuracy of 3-8 pixels. While they exclusively utilize event data, their hardware introduces a processing delay of up to 8 ms, which is significantly lower than the sensor's event rate. This poses the same data loss problem mentioned earlier, as it is still common in many commercial products. Additionally, it should be noted that the acquired pupil location from the model still requires further mapping to obtain the corresponding gaze point. Reference [48] combines DVS with a Coded Differential Lighting (CDL) method for glint tracking, achieving a sub-pixel accuracy and a sampling rate of 1000 Hz. However, their approach consumes around 100 mW of power for the additional LEDs, leading to higher energy consumption per estimation compared to our system. It also requires further mapping to get gaze points.

E. Mobile High-Performance Eye Tracking for AR/VR and Extended Use Cases

In this work, we explore advanced mobile high-performance eye tracking techniques designed to extend the applicability and impact of our system in AR/VR and various real-world scenarios. By leveraging event-driven processing and optimized hardware, we achieve sub-degree accuracy across a wide FoV, ensuring seamless integration into AR/VR headsets. Additionally, our system's low power consumption and minimal latency make it ideal for assistive healthcare devices, such as gaze-controlled communication tools for individuals with motor impairments. Adaptive algorithms dynamically adjust to varying eye shapes and sensor noise, enhancing robustness in real-world environments. These innovations extend the applicability of our system, making it versatile for both consumer electronics and medical devices, thereby improving accessibility and user interaction across diverse scenarios. This compact yet powerful approach highlights significant societal benefits and broadens the impact of our research.

VI. CONCLUSION

This work presents a fully-DVS-based event-driven low-power eye tracking processor designed and fabricated for AR/VR applications on wearable devices. A lightweight event-density-based method is proposed and implemented for event-driven and frame-less ROI determination directly on

AER data stream. An estimator with a streamlined output mapping strategy and an adaptive window-sliding convolution scheme is implemented for gaze estimation acceleration. Fabricated in UMC 40-nm LP-CMOS, the proposed processor has a total silicon area of 3.2 mm^2 and core area of 2.52 mm^2. Under a core voltage of 1.0 V, the processor can operate at 200 MHz frequency and deliver a dynamic frame rate up to 1.2 kHz with 12.7 μJ energy consumption per gaze estimation. The average estimation accuracy reaches 0.91° in a 96°×64° FoV. This work demonstrates advancements in event-based eye tracking systems, paving the way for more effective eye tracking solutions in wearable AR/VR devices.

REFERENCES

[1] I. B. Adhanom, P. MacNeilage, and E. Folmer, "Eye tracking in virtual reality: A broad review of applications and challenges," *Virtual Reality*, vol. 27, no. 2, pp. 1481–1505, Jun. 2023.

[2] P. Majaranta and A. Bulling, "Eye tracking and eye-based human–computer interaction," in *Advances in Physiological Computing*. Berlin, Germany: Springer, 2014, pp. 39–65.

[3] D. Jeong, M. Jeong, U. Yang, and K. Han, "Eyes on me: Investigating the role and influence of eye-tracking data on user modeling in virtual reality," *PLoS ONE*, vol. 17, no. 12, Dec. 2022, Art. no. e0278970.

[4] M. Yang, Y. Gao, L. Tang, J. Hou, and B. Hu, "Wearable eye-tracking system for synchronized multimodal data acquisition," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 34, no. 6, pp. 5146–5159, Jun. 2024.

[5] N. Menéndez and E. Bozkir, "Eye-tracking devices for virtual and augmented reality metaverse environments and their compatibility with the European union general data protection regulation," *Digit. Soc.*, vol. 3, no. 2, p. 39, Aug. 2024.

[6] J. J. MacInnes, S. Iqbal, J. Pearson, and E. N. Johnson, "Wearable eye-tracking for research: Automated dynamic gaze mapping and accuracy/precision comparisons across devices," *BioRxiv*, Jun. 2018, doi: 10.1101/299925.

[7] A. L. Gardony, R. W. Lindeman, and T. T. Brunyé, "Eye-tracking for human-centered mixed reality: Promises and challenges," *Proc. SPIE*, vol. 11310, pp. 230–247, Jul. 2020.

[8] A. Changwani and T. Sarode, "Low-cost eye tracking for foveated rendering using machine learning," in *Proc. IEEE Int. Conf. Cloud Comput. Emerg. Markets (CCEM)*, Sep. 2019, pp. 32–39.

[9] P. Lungaro, R. Sjoberg, A. J. F. Valero, A. Mittal, and K. Tollmar, "Gaze-aware streaming solutions for the next generation of mobile VR experiences," *IEEE Trans. Vis. Comput. Graph.*, vol. 24, no. 4, pp. 1535–1544, Apr. 2018.

[10] M. D. Binder, N. Hirokawa, and U. Windhorst, *Saccade, Saccadic Eye Movement*. Berlin, Heidelberg: Springer, 2009, p. 3557, doi: 10.1007/978-3-540-29678-2_5190.

[11] S. Doroudian, "Collaboration in immersive environments: Challenges and solutions," 2023, *arXiv:2311.00689*.

[12] N. Stein et al., "A comparison of eye tracking latencies among several commercial head-mounted displays," *i-Perception*, vol. 12, no. 1, Jan. 2021, Art. no. 2041669520983338.

[13] C. Brandli, R. Berner, M. Yang, S.-C. Liu, and T. Delbruck, "A 240×180 130 dB 3μs latency global shutter spatiotemporal vision sensor," *IEEE J. Solid-State Circuits*, vol. 49, no. 10, pp. 2333–2341, Oct. 2014.

[14] G. Gallego et al., "Event-based vision: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 1, pp. 154–180, Jan. 2022.

[15] Y. Zhang et al., "An event-driven spatiotemporal domain adaptation method for DVS gesture recognition," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 69, no. 3, pp. 1332–1336, Mar. 2022.

[16] Y. Hu, H. M. Chen, and T. Delbruck, "Slasher: Stadium racer car for event camera end-to-end learning autonomous driving experiments," in *Proc. IEEE Int. Conf. Artif. Intell. Circuits Syst. (AICAS)*, Mar. 2019, pp. 29–33.

[17] G. Chen, H. Cao, J. Conradt, H. Tang, F. Rohrbein, and A. Knoll, "Event-based neuromorphic vision for autonomous driving: A paradigm shift for bio-inspired visual sensing and perception," *IEEE Signal Process. Mag.*, vol. 37, no. 4, pp. 34–49, Jul. 2020.

[18] G. Chen et al., "A novel illumination-robust hand gesture recognition system with event-based neuromorphic vision sensor," *IEEE Trans. Autom. Sci. Eng.*, vol. 18, no. 2, pp. 508–520, Apr. 2021.

[19] A. N. Angelopoulos, J. N. P. Martel, A. P. Kohli, J. Conradt, and G. Wetzstein, "Event-based near-eye gaze tracking beyond 10,000 Hz," *IEEE Trans. Vis. Comput. Graph.*, vol. 27, no. 5, pp. 2577–2586, May 2021.

[20] N. Li, M. Chang, and A. Raychowdhury, "E-gaze: Gaze estimation with event camera," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 7, pp. 4796–4811, Jul. 2024.

[21] H. You et al., "EyeCoD: Eye tracking system acceleration via flatcam-based algorithm & hardware co-design," *IEEE Micro*, vol. 43, no. 3, pp. 88–97, Aug. 2023.

[22] L. Świrski and N. A. Dodgson, "A fully-automatic, temporal approach to single camera, glint-free 3D eye model fitting," in *Proc. PETMEI*, Aug. 2013, pp. 1–11.

[23] C.-C. Lai, S.-W. Shih, and Y.-P. Hung, "Hybrid method for 3-D gaze tracking using glint and contour features," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 25, no. 1, pp. 24–37, Jan. 2015.

[24] S. Hutton, "Eye tracking methodology," in *Eye Movement Research* (Studies in Neuroscience, Psychology and Behavioral Economics), C. Klein and U. Ettinger, Eds., Cham, Switzerland: Springer, 2019, pp. 277–308.

[25] C. Mestre, J. Gautier, and J. Pujol, "Robust eye tracking based on multiple corneal reflections for clinical applications," *J. Biomed. Opt.*, vol. 23, no. 3, Mar. 2018, Art. no. 035001, doi: 10.1117/1.JBO.23.3.035001.

[26] S. Ghosh, A. Dhall, M. Hayat, J. Knibbe, and Q. Ji, "Automatic gaze analysis: A survey of deep learning based approaches," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 1, pp. 61–84, Jan. 2024.

[27] K. I. Lee, J. H. Jeon, and B. C. Song, "Deep learning-based pupil center detection for fast and accurate eye tracking system," in *Proc. 16th Eur. Conf. Comput. Vis. (ECCV)*. Cham, Switzerland: Springer, Jan. 2020, pp. 36–52.

[28] J. Kim et al., "NVGaze: An anatomically-informed dataset for low-latency, near-eye gaze estimation," in *Proc. CHI Conf. Hum. Factors Comput. Syst.*, May 2019, pp. 1–12.

[29] B. Li, H. Fu, D. Wen, and W. Lo, "Etracker: A mobile gaze-tracking system with near-eye display based on a combined gaze-tracking algorithm," *Sensors*, vol. 18, no. 5, p. 1626, May 2018.

[30] P. L. Mazzeo, D. D'Amico, P. Spagnolo, and C. Distante, "Deep learning based eye gaze estimation and prediction," in *Proc. 6th Int. Conf. Smart Sustain. Technol. (SpliTech)*, Sep. 2021, pp. 1–6.

[31] T. Hirata et al., "A vision system with 1-inch 17-mpixel 1000-fps block-controlled coded-exposure stacked-CMOS image sensor for computational imaging and adaptive dynamic range control," *IEEE Open J. Circuits Syst.*, vol. 3, pp. 311–323, 2022.

[32] G. Zhao et al., "Ev-eye: Rethinking high-frequency eye tracking through the lenses of event cameras," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., Red Hook, NY, USA: Curran Associates, 2023, pp. 62169–62182. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/c41b5d8c1ba15b2aa83e4fa1541f02c8-Paper-Datasets_and_Benchmarks.pdf

[33] Z. Wang et al., "Event-based eye Tracking. AIS 2024 challenge survey," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2024, pp. 5810–5825.

[34] R. Eki et al., "A 1/2.3inch 12.3Mpixel with on-chip 4.97TOPS/W CNN processor back-illuminated stacked CMOS image sensor," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 154–156.

[35] C. Li, "Two-stream vision sensors," Ph.D. dissertation, ETH Zurich, Dept. Elect. Inf. Eng. (D-ITET), Zurich, Switzerland, Jun. 2017.

[36] A. Banerjee, S. S. Prasad, N. K. Mehta, H. Kumar, S. Saurav, and S. Singh, "Gaze detection using encoded retinomorphic events," in *Intelligent Human Computer Interaction*, H. Zaynidinov, M. Singh, U. S. Tiwary, and D. Singh, Eds., Cham, Switzerland: Springer, 2023, pp. 442–453.

[37] Q. Chen, Z. Wang, S.-C. Liu, and C. Gao, "3ET: Efficient event-based eye tracking using a change-based ConvLSTM network," in *Proc. IEEE Biomed. Circuits Syst. Conf. (BioCAS)*, Oct. 2023, pp. 1–5.

[38] A. K. Chaudhary et al., "RITnet: Real-time semantic segmentation of the eye for gaze tracking," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshop (ICCVW)*, Oct. 2019, pp. 3698–3702.

[39] D. P. Moeys et al., "Steering a predator robot using a mixed frame/event-driven convolutional neural network," in *Proc. 2nd Int. Conf. Event-Based Control, Commun., Signal Process. (EBCCSP)*, Jun. 2016, pp. 1–8.

[40] A. Paszke et al., "PyTorch: An imperative style,x high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, Jan. 2019, pp. 8024–8035.
[41] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
[42] Y. R. Pei, S. Brüers, S. Crouzet, D. McLelland, and O. Coenen, "A lightweight spatiotemporal network for online eye tracking with event camera," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2024, pp. 5780–5788. [Online]. Available: https://api.semanticscholar.org/CorpusID
[43] S. Tan, J. Huang, Q. Yan, L. Zheng, and Z. Zou, "A near-eye DVS-based end-to-end eye tracking processor for AR/VR applications," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2024, pp. 1–5.
[44] Y. Feng, N. Goulding-Hotta, A. Khan, H. Reyserhove, and Y. Zhu, "Real-time gaze tracking with event-driven eye segmentation," in *Proc. IEEE Conf. Virtual Reality 3D User Interfaces (VR)*, Mar. 2022, pp. 399–408.
[45] C. Farabet, B. Martini, B. Corda, P. Akselrod, E. Culurciello, and Y. LeCun, "NeuFlow: A runtime reconfigurable dataflow processor for vision," in *Proc. CVPR WORKSHOPS*, Jun. 2011, pp. 109–116.
[46] C. Palmero, A. Sharma, K. Behrendt, K. Krishnakumar, O. V. Komogortsev, and S. S. Talathi, "OpenEDS2020 challenge on gaze tracking for VR: Dataset and results," *Sensors*, vol. 21, no. 14, p. 4769, Jul. 2021.
[47] P. Bonazzi, S. Bian, G. Lippolis, Y. Li, S. Sheik, and M. Magno, "Retina: Low-power eye tracking with event camera and spiking hardware," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2024, pp. 5684–5692.
[48] T. Stoffregen, H. Daraei, C. Robinson, and A. Fix, "Event-based kilohertz eye tracking using coded differential lighting," in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. (WACV)*, Jan. 2022, pp. 3937–3945.
[49] Y. Zhao et al., "I-FlatCam: A 253 FPS, 91.49 μJ/frame ultra-compact intelligent lensless camera for real-time and efficient eye tracking in VR/AR," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2022, pp. 108–109.
[50] L. Chen et al., "Real-time gaze tracking with head-eye coordination for head-mounted displays," in *Proc. IEEE Int. Symp. Mixed Augmented Reality (ISMAR)*, Oct. 2022, pp. 82–91.

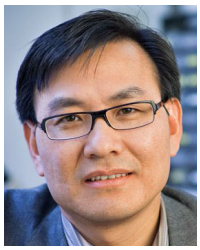
Shihang Tan received the B.E. degree in integrated circuit design and integrated system from Huazhong University of Science and Technology, Wuhan, China, in 2023. He is currently pursuing the M.S. degree with the School of Information Science and Technology, Fudan University, Shanghai, China. His research interests include energy-efficient microprocessor design for wearable devices and low-power circuit design.

Jinqiao Yang (Student Member, IEEE) received the B.E. degree in integrated circuit design and integrated systems from the University of Electronic Science and Technology of China, Chengdu, China. He is currently pursuing the Ph.D. degree with the School of Information Science and Technology, Fudan University, Shanghai. His research interests include energy-efficient spiking neural network processors and low-power architecture for neuromorphic computing.

Jiayu Huang received the B.E. degree in integrated circuit design and integrated system from the University of Electronic and Science and Technology of China, Chengdu, China, in 2021, and the M.Sc. degree in integrated circuit design and integrated system from the School of Information Science and Technology, Fudan University, Shanghai, China. His research interests include energy-efficient microprocessor design for AIoT and low-power circuit design.

Ziyi Yang received the B.E. degree from the University of Electronic Science and Technology of China, Chengdu, China. He is currently pursuing the Ph.D. degree with the School of Information Science and Technology, Fudan University, Shanghai. His research interests include neuromorphic hardware for brain-like computing and preprocessing of DVS event camera data.

Qinyu Chen (Member, IEEE) received the B.Eng. degree from Shandong University in 2016 and the Ph.D. degree from Nanjing University in 2021. She has been an Assistant Professor with Leiden Institute of Advanced Computer Science (LIACS), Leiden University, The Netherlands, since 2024. From 2019 to 2020, she was a Visiting Ph.D. Student, and from 2022 to 2024, a Post-Doctoral Researcher with the Sensors Group, Institute of Neuroinformatics, University of Zurich and ETH Zürich. Her research focuses on compact, energy-efficient, neuro-inspired intelligent circuits and systems for applications in healthcare, extended reality, and robotics. She received the 2022 BRIDGE Fellowship from the Swiss National Science Foundation (SNSF) and the Best Paper Award Honorable Mention from the IEEE Neural Systems and Applications Technical Committee (NSATC) in 2024. She serves on technical committees for IEEE NSATC (since 2022), CVPR AI for Streaming Workshop (2024), IEEE Standards Workshop on AI for Healthcare (2024), NICE Conference (2025), CVPR Event-Based Vision Workshop (2025), and as the Workshop/Tutorial Chair for FPL (2025).



Lirong Zheng (Senior Member, IEEE) received the Ph.D. degree in electronic system design from the KTH Royal Institute of Technology (KTH), Stockholm, Sweden, in 2001. He was with KTH as a Research Fellow, an Associate Professor, and a Full Professor. He is currently the Founding Director of the iPack VINN Excellence Center of Sweden and has been the Chair Professor in media electronics with KTH since 2006. He has been also a Guest Professor since 2008 and a Distinguished Professor since 2010 with Fudan University, Shanghai, China, where he currently holds the directorship of Shanghai Institute of Intelligent Electronics and Systems, Fudan University. He has authored more than 500 publications. His current research interests include electronic circuits, wireless sensors and systems for ambient intelligence, and the Internet of Things.

Zhuo Zou (Senior Member, IEEE) received the Ph.D. degree in electronic and computer systems from the KTH Royal Institute of Technology, Sweden, in 2012. Currently, he is with Fudan University, Shanghai, as a Full Professor, where he is conducting research on intelligent chips and systems for AIoT. Prior to joining Fudan, he was the Assistant Director and a Project Leader with the VINN iPack Excellence Center, KTH. His current research interests include low-power circuits, energy-efficient SoC, neuromorphic computing, and their applications in AIoT and autonomous systems. He has also been an Adjunct Professor and a Docent with the University of Turku, Finland. He is the Vice Chair of IFIP WG8.12.